

Авторизация: как это всё работает

Информационная система городской телефонной сети

Аутентификация, сессии, ролевая модель (RBAC) и защита — от cookie до SQL

Содержание

1	Картина целиком	2
2	Три кирпичика	2
2.1	Пароли — Argon2 (никогда не в открытом виде)	2
2.2	Сессии — серверные, через cookie	3
2.3	Права — RBAC (роли и разрешения)	3
3	Где это в базе данных	3
3.1	0007 — пользователи и RBAC	3
3.2	0012 — аккаунты абонентов	3
4	Вход сотрудника по шагам	3
5	Каждый защищённый запрос: экстрактор currentUser	4
5.1	Где проверяются права	4
6	Суперадмин «из коробки» (bootstrap)	5
7	Личный кабинет абонента — авторизация по владению	5
8	Фронтенд: сторы, guard и UI по правам	5
8.1	Два стора (Pinia)	5
8.2	Сторож маршрутов (router.beforeEach)	6
8.3	Cookie и ошибки в HTTP-клиенте	6
9	Выход	6
10	Полный сквозной сценарий	7
11	Шпаргалка к защите (по авторизации)	7

1 Картина целиком

В системе **две независимые «двери»** для входа — у них разные пользователи, разные таблицы и разные права, но **один** механизм сессий:

Кто входит	Сотрудники (операторы/админ)	Абоненты (горожане)
Таблица в БД	app_user	customer
Точки входа	/api/auth/login, /logout, /me	/api/portal/register, /login, /logout, /me
Что определяет доступ	Роли и права (RBAC)	Владение своими данными
Где на сайте	/staff/login → операторская панель	/portal/login → личный кабинет
Ключ в сессии	user_id	customer_id

Различают два понятия:

- **Аутентификация** (authentication) — «кто ты»: проверка логина и пароля, после чего заводится **сессия**.
- **Авторизация** (authorization) — «что тебе можно»: у сотрудника решают **права** (через роли), у абонента — **принадлежность** записи именно ему.

Где смотреть.

- Бэкенд сотрудников: backend/server/src/auth/ (routes.rs, user.rs, password.rs, bootstrap.rs).
- Бэкенд абонентов: backend/server/src/portal.rs.
- Проверки прав в эндпоинтах: crud.rs, admin.rs, analytics.rs, raw_query.rs, settings.rs.
- Фронт: frontend/src/stores/auth.ts, stores/customer.ts, router/index.ts, api/client.ts.
- Схема БД: migrations/0007_auth_rbac.sql (RBAC), migrations/0012_customer_portal.sql (абоненты).

2 Три кирпичика

Вся авторизация собрана из трёх простых механизмов.

2.1 Пароли — Argon2 (никогда не в открытом виде)

Пароль в БД хранится не как текст, а как **хеш** Argon2 (современный, устойчивый к перебору алгоритм). При входе пароль не «расшифровывают» — заново хешируют введённое и сравнивают.

```
// auth/password.rs
pub fn hash_password(password: &str) -> anyhow::Result<String> {
    let salt = SaltString::generate(&mut OsRng);           // случайная «соль»
    let hash = Argon2::default()
        .hash_password(password.as_bytes(), &salt)?.to_string();
    Ok(hash)                                               // "$argon2id$v=19$m=...$..."
}

pub fn verify_password(password: &str, hash: &str) -> bool {
    match PasswordHash::new(hash) {
        Ok(parsed) => Argon2::default()
            .verify_password(password.as_bytes(), &parsed).is_ok(),
        Err(_) => false,
    }
}
```

- **Соль** (salt) — случайная добавка к каждому паролю: одинаковые пароли дают разные хеши, готовые «радужные таблицы» бесполезны.
- Хеширует **только** бэкенд. В БД (password_hash TEXT) и тем более на фронт пароль никогда не попадает.

2.2 Сессии — серверные, через cookie

После успешного входа сервер заводит **сессию** и кладёт в неё id пользователя. Браузеру уходит **cookie** с непредсказуемым идентификатором сессии; **сами данные** сессии лежат в PostgreSQL (а не в cookie).

```
// main.rs — слой сессий поверх всех роутов
let session_store = PostgresStore::new(pool.clone());
session_store.migrate().await?; // создаёт таблицу сессий
let session_layer = SessionManagerLayer::new(session_store)
    .with_secure(false) // dev: разрешить http
    .with_expiry(Expiry::OnInactivity(time::Duration::days(7)));
```

- **tower-sessions** + хранилище в Postgres: на каждый запрос слой по cookie находит сессию и даёт обработчику объект Session.
- Expiry::OnInactivity(7 дней) — сессия живёт, пока ею пользуются; неделя простоя — и она протухает.
- Cookie обычно HttpOnly (JavaScript её не читает — защита от XSS-кражи), что хорошо.

2.3 Права — RBAC (роли и разрешения)

У сотрудников доступ решает **ролевая модель** (Role-Based Access Control): пользователю выдают **роли**, в ролях лежат **права**. Это требование «роли не прибиты гвоздями в коде» — всё лежит в БД и настраивается суперадмином.

3 Где это в базе данных

3.1 0007 — пользователи и RBAC

```
CREATE TABLE app_user (
    id BIGSERIAL PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL, -- Argon2-хеш
    is_superuser BOOLEAN NOT NULL DEFAULT FALSE, -- обходит проверки прав
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    last_login_at TIMESTAMPTZ
);
CREATE TABLE permission ( id BIGSERIAL PRIMARY KEY, code TEXT NOT NULL UNIQUE, ... );
CREATE TABLE role ( id BIGSERIAL PRIMARY KEY, name TEXT NOT NULL UNIQUE, ... );
CREATE TABLE role_permission ( role_id ... REFERENCES role, permission_id ... REFERENCES
permission,
                                PRIMARY KEY (role_id, permission_id) );
CREATE TABLE user_role ( user_id ... REFERENCES app_user, role_id ... REFERENCES role,
                           PRIMARY KEY (user_id, role_id) );
```

- permission.code — право вида сущность:действие (subscriber:read, invoice:create) + спец-права analytics:read, raw_query:run, rbac:manage.
- role_permission и user_role — **таблицы-связки** многие-ко-многим: «роли ↔ права» и «пользователи ↔ роли». Составной первичный ключ не даёт задвоить пару.
- Готовые роли (0010): **superadmin** (всё), **operator** (чтение/создание/изменение домена + аналитика + raw-query, без управления людьми), **viewer** (только чтение).

3.2 0012 — аккаунты абонентов

customer — отдельная таблица (логин/пароль + ФИО, категория). Это **не** app_user: горожанин и сотрудник — разные миры. У subscriber и installation_queue есть customer_id — так линия/заявка привязана к тому, кто её подал.

4 Вход сотрудника по шагам

```
// auth/routes.rs — POST /api/auth/login
let row = sqlx::query_as("SELECT id, password_hash FROM app_user
                        WHERE username = $1 AND is_active")
    .bind(&input.username).fetch_optional(pool).await?;
let Some((id, hash)) = row else { return Err(AppError::InvalidCredentials) };
if !verify_password(&input.password, &hash) { return Err(AppError::InvalidCredentials) }
```

```

session.insert(USER_ID_KEY, id).await?; // USER_ID_KEY = "user_id"
sqlx::query("UPDATE app_user SET last_login_at = now() WHERE id = $1").bind(id)...;
let user = load_current_user(pool, id).await?.ok_or(AppError::Unauthorized)?;
Ok(Json(user)) // + cookie сессии в ответе

```

1. Ищем активного пользователя по логину. Нет такого или пароль не сошёлся → 401 с текстом «Неверный логин или пароль» (AppError::InvalidCredentials).
2. Кладём user_id в сессию. Слой сессий ставит браузеру cookie.
3. Обновляем last_login_at и возвращаем профиль с правами (см. ниже) — фронт сразу знает, что показывать.

Важно. Логин и «не вошёл» — разные ответы. Неверные логин/пароль → 401 «Неверный логин или пароль»; запрос без сессии к защищённому ресурсу → 401 «Требуется авторизация». Оба 401, но текст разный.

5 Каждый защищённый запрос: экстрактор CurrentUser

Главная «магия» — тип CurrentUser реализует FromRequestParts. Стоит написать его аргументом обработчика — и Ахум **до** входа в функцию достанет сессию, найдёт пользователя и его права. Нет сессии — обработчик даже не вызовется, сразу 401.

```

// auth/user.rs
impl FromRequestParts<S> for CurrentUser {
    async fn from_request_parts(parts, state) -> Result<Self, AppError> {
        let session = Session::from_request_parts(parts, state).await
            .map_err(|_| AppError::Unauthorized)?;
        let user_id: i64 = session.get(USER_ID_KEY).await?
            .ok_or(AppError::Unauthorized)?; // нет user_id → 401
        load_current_user(pool, user_id).await?
            .ok_or(AppError::Unauthorized) // удалён/деактивирован → 401
    }
}

```

load_current_user собирает профиль и **плоский набор прав** одним JOIN-ом по связкам:

```

SELECT DISTINCT p.code
FROM user_role ur
JOIN role_permission rp ON rp.role_id = ur.role_id
JOIN permission p      ON p.id = rp.permission_id
WHERE ur.user_id = $1;

```

Проверка права — два метода на CurrentUser:

```

pub fn has(&self, perm: &str) -> bool {
    self.is_superuser || self.permissions.contains(perm) // суперадмин — мимо проверок
}
pub fn require(&self, perm: &str) -> AppResult<()> {
    if self.has(perm) { Ok(()) } else { Err(AppError::Forbidden) } // иначе 403
}

```

5.1 Где проверяются права

Один обобщённый CRUD проверяет право по имени сущности и действию — без дублирования:

```

// crud.rs — на каждый из 5 обработчиков
async fn create<E>(user: CurrentUser, ...) -> AppResult<...> {
    user.require(&format!("{}", create", E::NAME)); // напр. "subscriber:create"
    ...
}

```

Эндпоинт	Требуемое право
CRUD GET /api/<сущность>	<сущность>:read
CRUD POST/PUT/DELETE	<сущность>:create / :update / :delete
Аналитика GET /api/analytics/*	analytics:read
SQL-консоль POST /api/raw-query	raw_query:run
Настройки биллинга	billing_settings:read / :update
Список ролей/прав/юзеров	role:read / user:read
Правка ролей/юзеров	role:create/update/delete,user:...
Выдать роли права / юзеру роли	rbac:manage

Важно. Итог: даже если кто-то в обход фронта дёрнет API напрямую, сервер всё равно потребует сессию и нужное право. Доступ **не** держится на «спрятанных кнопках».

6 Суперадмин «из коробки» (bootstrap)

При старте сервер проверяет, есть ли суперадмин, и если нет — создаёт его из конфига. Иначе в свежую систему было бы некому войти.

```
// auth/bootstrap.rs — вызывается из main.rs при запуске
if /* нет app_user с этим username */ {
  let hash = hash_password(&cfg.superadmin_password?);
  // INSERT app_user (... is_superadmin = TRUE ...)
  // INSERT user_role SELECT uid, id FROM role WHERE name = 'superadmin'
}
```

Логин/пароль берутся из [auth] в backend/config.toml (по умолчанию **admin / admin**) или из переменных окружения GTS__AUTH__....

7 Личный кабинет абонента — авторизация по владению

У портала **нет** ролей. Логика «что можно» решает иначе: экстрактор CurrentCustomer определяет, **кто** вошёл (по customer_id в сессии), а каждое действие проверяет, что запись **принадлежит** этому абоненту.

```
// portal.rs — устройство то же, что у CurrentUser, но ключ другой
const CUSTOMER_ID_KEY: &str = "customer_id";
// ... session.get(CUSTOMER_ID_KEY) -> ok_or(AppError::Unauthorized)

// проверка владения линией перед операцией над ней
async fn owned_subscriber(pool, customer_id, number_id) -> AppResult<i64> {
  let row = sqlx::query_as("SELECT s.id FROM subscriber s
    WHERE s.customer_id = $1 AND s.phone_number_id = $2")
    .bind(customer_id).bind(number_id).fetch_optional(pool).await?;
  row.map(|(id,)| id).ok_or(AppError::Forbidden) // чужая линия → 403
}
```

Так абонент может включить межгород, позвонить или оплатить счёт **только** по своим линиям. Регистрация (/portal/register) сама заводит сессию — сразу после неё человек «внутри».

8 Фронтенд: сторы, guard и UI по правам

8.1 Два стора (Pinia)

Состояние входа держат два независимых стора. auth (сотрудник) знает права и умеет can():

```
// stores/auth.ts
getters: {
```

```

    isAuthenticated: (s) => !!s.user,
    can: (s) => (perm) => !!s.user && (s.user.is_superuseradmin ||
s.user.permissions.includes(perm)),
  },
  actions: {
    async fetchMe() { try { this.user = (await api.get('/auth/me')).data } catch { this.user
= null } ... },
    async login(u, p) { this.user = (await api.post('/auth/login', { username: u, password:
p })).data },
    async logout() { await api.post('/auth/logout'); this.user = null },
  }

```

customer (абонент) устроен так же, но ходит в /portal/* и хранит профиль абонента. fetchMe нужен, чтобы при перезагрузке страницы понять, есть ли ещё живая сессия (cookie уже в браузере).

8.2 Сторож маршрутов (router.beforeEach)

Перед каждым переходом guard пускает публичные страницы, а для защищённых — проверяет вход в нужном «мире» и право на конкретную страницу:

```

router.beforeEach(async (to) => {
  if (to.meta.public) return true // лендинг, /staff/login, /portal/
  login
  if (to.meta.area === 'portal') { // личный кабинет
    const c = useCustomerStore(); if (!c.loaded) await c.fetchMe()
    if (!c.isAuthenticated) return { name: 'portal-login', query: { redirect:
to.fullPath } }
    return true
  }
  const auth = useAuthStore(); if (!auth.loaded) await auth.fetchMe() // операторская
часть
  if (!auth.isAuthenticated) return { name: 'staff-login', query: { redirect:
to.fullPath } }
  const perm = to.meta.perm // напр. 'analytics:read'
  if (perm && !auth.can(perm)) return { name: 'dashboard' } // нет права → на
главную
  return true
})

```

- query: { redirect: ... } — после входа возвращаемся туда, куда шли.
- У страниц с правами стоит meta: { perm: '...' } (аналитика, SQL-консоль, настройки, админка). Пункты меню тоже скрыты через v-if="auth.can('...')" — но это лишь **удобство**: настоящая защита на сервере.

8.3 Cookie и ошибки в HTTP-клиенте

```
// api/client.ts
const api = axios.create({ baseURL: '/api', withCredentials: true }) // слать cookie сессии
```

withCredentials: true обязателен — иначе браузер не пошлёт cookie и каждый запрос будет «не авторизован». Тот же клиент превращает ответы об ошибке в человеко-читаемый русский текст: 401 → «Требуется авторизация», 403 → «Доступ запрещён».

9 Выход

```

async fn logout(session: Session) -> AppResult<Json<Value>> {
  session.flush().await?; // удалить сессию целиком
  Ok(Json(json!({ "status": "ok" })))
}

```

flush() стирает серверную сессию (cookie становится бесполезной), а стор на фронте обнуляет user/customer. Выходы из кабинета и из панели независимы.

10 Полный сквозной сценарий

1. Браузер --POST /api/auth/login {admin, admin}--> Сервер
2. Сервер: SELECT app_user ... ; verify_password (Argon2) ; session.insert("user_id", 7)
3. Сервер --200 + Set-Cookie: id=<случайный>--> Браузер (профиль + права в теле)
4. Браузер --GET /api/subscribers (Cookie: id=...)-----> Сервер
5. Сервер: экстрактор CurrentUser -> сессия -> user_id=7 -> грузит права
require("subscriber:read") -> ок (или 403)
6. Сервер --200 страница данных--> Браузер
- ...
7. Браузер --POST /api/auth/logout--> Сервер: session.flush() (сессия мертва)

11 Шпаргалка к защите (по авторизации)

Вопрос	Короткий ответ + где код
Где хранится пароль?	Только Argon2-хеш в app_user.password_hash / customer.password_hash.auth/password.rs.
Как держится «вход»?	Серверная сессия (tower-sessions + Postgres); в cookie — лишь id сессии.main.rs.
Как сервер узнаёт пользователя?	Экстрактор CurrentUser/CurrentCustomer читает сессию на каждом запросе.auth/user.rs,portal.rs.
Что такое право?	Строка сущность:действие в таблице permission; роли связывают права с пользователями.
Где проверяются права?	user.require("...") в каждом обработчике: crud.rs, admin.rs,analytics.rs,raw_query.rs,settings.rs.
Зачем суперадмин?	is_superuser обходит проверки; создаётся при старте из конфига (admin/admin).auth/bootstrap.rs.
Чем отличается портал?	Нет ролей: абонент видит/меняет только свои записи (owned_subscriber → 403).portal.rs.
Защита только на фронте?	Нет. Guard и v-if — удобство; решает всё сервер (сессия + право).
Коды ошибок?	Нет/неверная сессия → 401, нет права/чужое → 403. Мэппинг в error.rs.

Безопасность. Что **осознанно упрощено** для учебного проекта (в проде поменять):

- with_secure(false) — cookie ходит по http; в проде нужен https и secure-cookie.
- Дефолтные admin / admin и session_secret-заглушка в config.toml — заменить реальными секретами (через окружение GTS__AUTH__...).
- Нет ограничения числа попыток входа (rate-limit) и сложности пароля (минимум — 4 символа).

См. также: разбор CRUD — docs/crud.pdf, миграции — docs/migrations.pdf, общий разбор — docs/guide.pdf.